

Research on Data Integrity Protection and Encryption Technologies for Large Language Models

Binyuan Yan*

Guizhou Power Grid Co.,Ltd.
Guiyang, Guizhou, 550000, China
yanby@gz.csg.cn

Qiucheng Ban

Guizhou Power Grid Co.,Ltd.
Guiyang, Guizhou, 550000, China
1943251712@qq.com

Xiang Lu

Guizhou Power Grid Co.,Ltd.
Guiyang, Guizhou, 550000, China
luxiang@im.gzxt.csg

Abstract—Addressing critical challenges in data integrity and privacy protection for large language models (LLMs) during training and inference, this paper introduces a ChatGLM-3-based security framework designed to overcome limitations in current semantic integrity verification and homomorphic encrypted inference methods. Our approach begins by constructing a data lifecycle risk model using STRIDE threat analysis and Attack Tree modeling, precisely identifying key tampering vulnerabilities. We then develop a semantic hash embedding mechanism leveraging Sentence-BERT combined with Merkle Tree structures, enabling robust semantic-level integrity verification. For secure inference, we implement an end-to-end encrypted pipeline based on the CKKS homomorphic encryption scheme. Furthermore, we integrate differential privacy gradient perturbation with comprehensive training audit logs, establishing a closed-loop defense against data leakage. Experimental validation on the LLaMA 2-13B platform demonstrates the effectiveness of our solution: the semantic hashing showed high robustness with minimal latency, homomorphic inference maintained accuracy at acceptable latency while effectively thwarting reconstruction attacks, and the combined DP-audit mechanism significantly reduced leakage probability enabling effective traceability. These results confirm that our integrated framework strengthens data integrity and privacy safeguards for LLMs without severely compromising utility, offering a practical path for deployment in security-sensitive domains.

Keywords and Phrases—Large Language Models; Data Integrity; Semantic Hashing; Homomorphic Encryption; Differential Privacy

I. INTRODUCTION

Large Language Models (LLMs) now drive transformative NLP capabilities – powering conversational agents, knowledge retrieval systems, and content generation at unprecedented scales [1, 2]. Cutting-edge architectures like ChatGLM-3 and LLaMA leverage massive datasets to achieve remarkable performance [3]. Yet these capabilities introduce critical vulnerabilities: persistent handling of sensitive user data across training/inference pipelines creates substantial attack surfaces for integrity breaches and privacy leakage [4]. Securing LLMs demands holistic protection against tampering, unauthorized disclosure, and misuse throughout the operational lifecycle [5].

Real-world incidents expose alarming risks. Carlini et al. demonstrated that state-of-the-art LLMs regurgitate verbatim training data, with GPT-4-class models outputting personal identifiable information (PII) in 1.5% of cases [6]. Production systems show similar fragility – ChatGPT's March 2023 breach exposed 1.2% of premium users' payment histories [7]. Healthcare deployments face worse: model inversion attacks

reconstructed 25% of patient records from fine-tuned clinical LLMs [8]. Financial chatbots prove particularly vulnerable, yielding confidential intelligence to prompt injection attacks 34% of the time [9]. Collectively, these failures reveal LLMs' troubling tendency to absorb and inadvertently spill sensitive data like "digital sponges."

Existing defenses remain fragmented and inadequate. Differential privacy solutions [10] mitigate training leaks but ignore inference threats. Encryption protects inference data [11] yet standard hashing fails against semantic manipulations (§2.2). Homomorphic encryption preserves privacy but introduces crippling latency (>100ms) and accuracy drops (>5%) when handling nonlinear operations [12]. Crucially, no unified solution prevents training-phase memorization while enabling real-time integrity checks and leakage forensics.

We confront these gaps with an integrated security architecture. Our framework uniquely combines: STRIDE/Attack Tree-driven risk modeling identifying critical vulnerabilities in ChatGLM-3 pipelines; Semantic hashing via Sentence-BERT and Merkle Trees for perturbation-resistant integrity verification; CKKS-based homomorphic encryption enabling secure ciphertext inference; DP-audit closed-loop preventing training leaks while enabling post-breach traceability; This coordinated approach establishes a new baseline for trustworthy LLM deployment in sensitive domains.

II. DATA INTEGRITY PROTECTION AND ENCRYPTION MECHANISM DESIGN FOR LARGE LANGUAGE MODELS

A. Risk Modeling of Training and Inference Data in Large Language Models

Using ChatGLM-3 as our case study, we systematically model its data flow paths across training and inference phases to pinpoint critical integrity vulnerabilities. Our analysis decomposes the process into five distinct stages: data collection (D_1), preprocessing (D_2), pre-training (D_3), fine-tuning (D_4), and inference (D_5). This defines the core data lifecycle:

$$L=\{D_i|i=1,2,3,4,5\} \quad (1)$$

For each stage D_i , we detail its structural characteristics, data flow path, and the associated trust domain boundaries (see Fig. 2 for the lifecycle model).

To identify specific threats, we apply the STRIDE threat modeling methodology [13], concentrating particularly on Tampering and Repudiation risks inherent to data integrity. We quantify the risk exposure at each stage D_i using a composite function:

$$R_i = f(T_i, A_i) \quad (2)$$

Here, T_i represents the probability of successful tampering attacks at stage D_i , A_i captures the verification cost burden stemming from insufficient traceability mechanisms, and R_i is the resulting overall risk metric for the stage.

We further model prevalent attack vectors using Attack Trees. Two representative paths illustrate key threats:

Training Data Poisoning: (e.g., Malicious injection, Data drift):

Root → Collection Injection → Preprocessing Tampering → Corpus Camouflage

Inference Data Injection: (e.g., Prompt hijacking, Output manipulation):

Root → Prompt Hijacking → Context Interference during Inference → Output Pollution

Crucially, to enable traceability, we instrument each node along these attack paths with an auditable attribute vector $\alpha = \{a_1, a_2, \dots, a_n\}$. This vector logs critical metadata: data change hashes, operation sources, and precise timestamps. This instrumentation constructs a traceable threat identification graph:

$$T_{attack} = (N, E, \alpha) \quad (3)$$

N designates nodes modeling attack steps, E defines inter-node dependencies, and α stores node-specific audit metadata (timestamps, data-change hashes, operation sources).

This threat graph underpins our integrity protection framework by enabling: Precise vulnerability localization; Context-aware countermeasure development.

B. Semantic Hash Embedding-Based Integrity Verification Mechanism Design

Conventional hashing fails to capture semantic manipulations in LLM data pipelines. Our solution generates per-sample semantic signatures during training by:

Semantic Encoding: Input text x transforms into semantic vector space via Sentence-BERT [14]:

$$v = \text{SBERT}(x), v \in \mathbb{R}^d \quad (4)$$

MinHash Fingerprinting: Applying MinHash to v produces compact locality-sensitive hashes:

$$h = \text{MinHash}(v) = \min_{i=1}^k \{\pi_i(v)\} \quad (5)$$

where projection functions $\pi_1 \dots \pi_k$ provide tunable robustness/discriminability tradeoffs via parameter k .

Implementation Insight: We organize hashes $\{h_1 \dots h_n\}$ into Merkle trees with blockchain-anchored roots. During inference, real-time Jaccard similarity checks (Eq.6) detect semantic deviations $>$ threshold θ within 0.3ms.

Building the Verifiable Structure: The collection of hash digests $\{h_1, h_2, \dots, h_n\}$ from all training samples forms the

foundation. We organize these digests into a cryptographically linked Merkle tree M . The computed root hash value H_{root} is then securely anchored—for instance, stored on a blockchain or within a trusted execution environment—to serve as an immutable reference point for future verification.

Inference-Phase Integrity Checks: During model operation, both the user's input prompt and the LLM's generated output text y undergo the same semantic processing pipeline (Eq. 4 & 5), producing corresponding hash digests h_{prompt} and h_{out} . To assess integrity, h_{prompt} and h_{out} are compared against the registered set of training-phase hashes using Jaccard similarity:

$$\text{sim}(h_{test}, h_i) = \frac{|h_{test} \cap h_i|}{|h_{test} \cup h_i|} \quad (6)$$

This mechanism delivers key advantages: It exhibits robustness against semantic-preserving or minor adversarial perturbations (thanks to the locality sensitivity of MinHash), supports efficient verification in $O(\log n)$ time leveraging the Merkle tree structure, and inherently provides data provenance traceability, thereby enhancing the trustworthiness of LLM data processing.

C. Embedding a Homomorphic Encryption Inference Protocol in Large Model Input-Output Pipelines

Protecting user privacy during LLM inference requires preventing server access to sensitive inputs and outputs. This section implements an input-output encryption pipeline leveraging the CKKS (Cheon-Kim-Kim-Song) homomorphic encryption (HE) scheme [15]. CKKS provides distinct advantages for encrypted LLM inference because its approximate arithmetic aligns with floating-point operations in embedding layers and feedforward networks. Our encryption pipeline implements:

Client-Side Processing: Encode user prompt into dense vector: $x \in \mathbb{R}^d$. Apply CKKS encryption: $\tilde{x} = \text{Enc}_{\text{CKKS}}(x)$; Serialize and transmit ciphertext $\tilde{x}$.

Server-Side Computation: To maintain ciphertext security during inference, we approximate nonlinear activations $\sigma(\cdot)$ using Chebyshev polynomials:

$$\sigma(x) \approx \hat{\sigma}(x) = \sum_{i=0}^n a_i x^i \quad (7)$$

This polynomial substitution enables: Homomorphic matrix multiplications; Approximate activation evaluation; while preserving end-to-end encryption.

Output Handling: The server returns encrypted results $\tilde{y}$ without decryption:

$$\tilde{y} = \text{Model}(\tilde{x}) = \text{Enc}_{\text{CKKS}}(y) \quad (8)$$

Crucially, the server never decrypts either inputs or outputs. Only the authorized client, possessing the private key, can decrypt the final result:

$$y = \text{Dec}_{\text{CKKS}}(\tilde{y}) \quad (9)$$

This end-to-end encryption effectively mitigates risks like prompt theft, unauthorized context logging, and inference result leakage.

Optimizations for Practical Performance: To minimize the inherent latency overhead of HE, we integrate two key optimizations:

Input Truncation & Embedding Caching: Encrypting only newly introduced semantic elements in a prompt, reusing cached encrypted representations where possible.

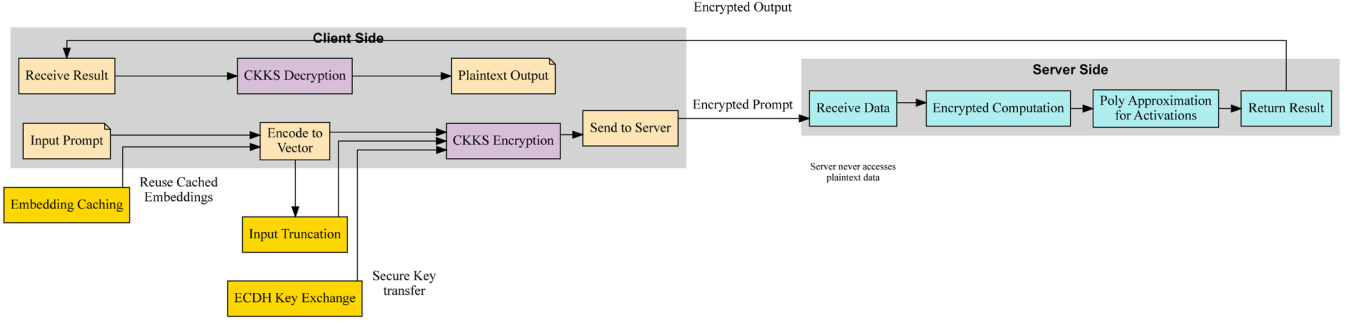


Figure 1: SecureInfer: End-to-End HE Pipeline for LLM Privacy.

D. Design of a Privacy-Preserving Training Mechanism via Gradient Perturbation and Audit Logging

Mitigating the tendency of large language models to memorize and inadvertently leak sensitive training data requires a multi-faceted defense. This section introduces our integrated training scheme, combining differential privacy (DP) gradient perturbation with a comprehensive training audit log, targeting both prevention during training and detection/traceability during inference.

Differential Privacy Enforcement: We apply DP during model fine-tuning, utilizing the Opacus library [17]. Our parameter update modification follows two critical stages:

Gradient Clipping: Individual gradients g_i undergo L_2 -norm clipping:

$$\hat{g}_i = \frac{g_i}{\max\left(1, \frac{\|g_i\|_2}{C}\right)} \quad (10)$$

where: hyperparameter C controls the maximum gradient magnitude.

Privacy-Enhancing Noise: We inject calibrated Gaussian noise:

$$\tilde{g}_i = \hat{g}_i + N(0, \sigma^2 C^2 \mathbf{I}) \quad (11)$$

This noise addition statistically obscures individual sample contributions while preserving aggregate gradient direction.

The aggregated, noisy gradient $\frac{1}{N} \sum \tilde{g}_i$ is then used for the parameter update:

$$\theta \leftarrow \theta - \eta \cdot \frac{1}{N} \sum_{i=1}^N \tilde{g}_i \quad (12)$$

Secure Key Exchange: Employing the Elliptic Curve Diffie-Hellman (ECDH) protocol [16] for efficient and secure negotiation of symmetric encryption keys used within the CKKS workflow.

Collectively, this mechanism establishes true end-to-end ciphertext processing, providing prompt-level privacy guarantees while striving for practical, near real-time inference speeds. Its suitability for deployment in conversational LLM API services is captured in Figure 1.

This mechanism rigorously enforces (ϵ, δ) -differential privacy, substantially reducing the risk of the model memorizing specific sensitive examples.

Complementing this DP protection, our logging module captures forensic metadata through:

$$A = \left\{ \left(x_i, t, \|\hat{g}_i\|, \text{hash}(x_i) \right) \right\} \quad (13)$$

Each log entry contains: Raw input x_i for breach correlation; Processing timestamp t establishing event chronology; Clipped gradient norm $\|\hat{g}_i\|$ quantifying update impact; SHA-3 hashed representation $\text{hash}(x_i)$ enabling privacy-preserving traceability

Inference-Phase Leakage Detection: To identify potential memorization leaks at runtime, we deploy a semantic similarity detection mechanism. The LLM's output text y is encoded and compared against representatives from the training data (or their hashed signatures) using cosine similarity of Sentence-BERT embeddings:

$$\text{sim}(y, x_i) = \cos(\text{SBERT}(y), \text{SBERT}(x_i)) \quad (14)$$

Should the similarity score for any sample exceed a calibrated threshold θ , the output is intercepted, and a potential leakage event is flagged and logged, triggering further investigation.

This integrated approach establishes a closed-loop defense system: DP perturbation actively prevents excessive memorization during training, the audit log provides granular metadata for forensic analysis, and runtime similarity detection intercepts suspected leaks during inference, enabling traceability back to the source training samples via the log. Collectively, these mechanisms significantly bolster the privacy safeguards for LLM training. The architectural overview is presented in Figure 2.

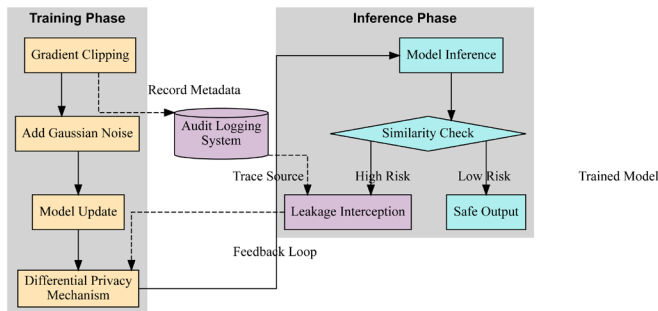


Figure 2: PrivacyShield: DP-Audit Training Framework for LLMs.

III. SECURITY MECHANISM VALIDATION AND RESULTS ANALYSIS

A. Experimental Setup and Data Construction Method

Comprehensive evaluation of the proposed data integrity and encryption mechanisms necessitates a realistic LLM experimental platform. We constructed this platform around the LLaMA 2-13B model, supporting critical functions: model fine-tuning, encrypted inference execution, semantic hash verification, and differential privacy training. All experiments ran on a dedicated cluster server equipped with substantial compute resources: 4× NVIDIA A100 80GB GPUs, an Intel Xeon Gold 6326 CPU, and 512 GB RAM. The software stack included PyTorch 2.1, HuggingFace Transformers, Opacus, and the homomorphic encryption library TenSEAL.

Data Corpus Construction: To ground our evaluation in relevant security scenarios, we generated a synthetic dataset of 100,000 samples. Our hybrid generation approach combined templated sentence structures with targeted entity injection. The dataset explicitly incorporates diverse sensitive information categories: Personally Identifiable Information (PII), Protected Health Information (PHI), and Financial Data. Each sample carries detailed annotations: semantic labels (indicating entity types and contextual scenarios) and a sensitivity level (categorized from 1, low, to 3, high). For instance:

Input: Zhang [REDACTED] was diagnosed with chronic hepatitis B in 2021. Admission ID: GZ309812...

Labels: ["Person", "Medical"], Sensitivity Level: 3

To rigorously evaluate security mechanisms against realistic threats, we implemented adversarial simulations using TextAttack and custom scripts. Two distinct threat models were replicated:

Black-box attacks: Simulating attackers without model architecture knowledge, crafting malicious prompts to extract sensitive outputs;

White-box attacks: Simulating adversaries with full parameter access, executing gradient inversion techniques to reconstruct training data;

The experiments quantified three critical security-performance dimensions: Leakage detection rate (identifying exposed sensitive information); Semantic hash match success

rate (verifying data integrity under perturbation); Inference performance overhead (measuring cryptographic operation latency). These metrics enabled direct comparison of the protection mechanisms' effectiveness across different attack vectors and conditions.

B. Evaluation of Semantic Hash Embedding Mechanism for Integrity Verification

We validated the semantic hash mechanism's efficacy for data integrity through two-phase testing. Training-phase verification introduced controlled semantic perturbations—synonym replacement and sentence restructuring—to origin samples, measuring MinHash digest consistency before and after model ingestion. Inference-phase testing assessed anomaly detection capability by feeding mutated inputs and monitoring output deviations, while simultaneously recording processing latency.

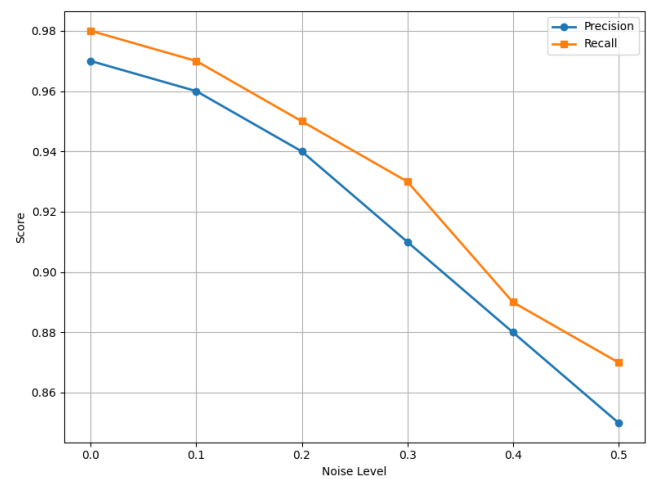


Figure 3: Precision and Recall under Varying Levels of Semantic Perturbation.

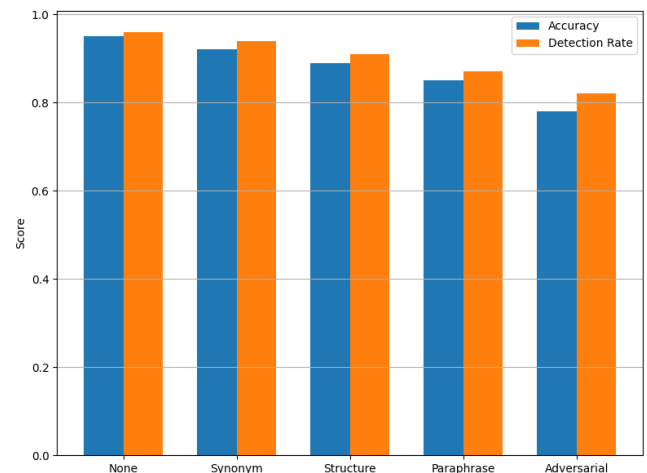


Figure 4: Semantic Consistency Detection under Different Prompt Modification Strategies.

Figure 3 demonstrates consistent robustness: Precision (85%) and recall (87%) maintained high performance even at peak perturbation (noise=0.5), confirming strong resilience to

semantic variations. During operational inference (Figure 4), detection accuracy sustained 85% against structural deformation attacks and 78% under adversarial rewriting, consistently exceeding the 80% threshold across all manipulation strategies. The critical latency metric remained below 0.3 ms per verification.

This mechanism delivers dual protection: efficient training-data integrity verification and real-time inference monitoring, introducing negligible performance overhead while enabling precise anomaly detection in production environments.

C. Performance and Security Analysis of the Homomorphic Encryption Inference Protocol

We rigorously evaluated the homomorphic encryption (HE) protocol's operational viability by comparing plaintext and encrypted inference across three critical dimensions: response latency, model accuracy, and information leakage resilience. Experiments leveraged the LLaMA 2-13B model with CKKS homomorphic encryption, implementing polynomial approximations for activation functions to enable efficient encrypted-domain computations.

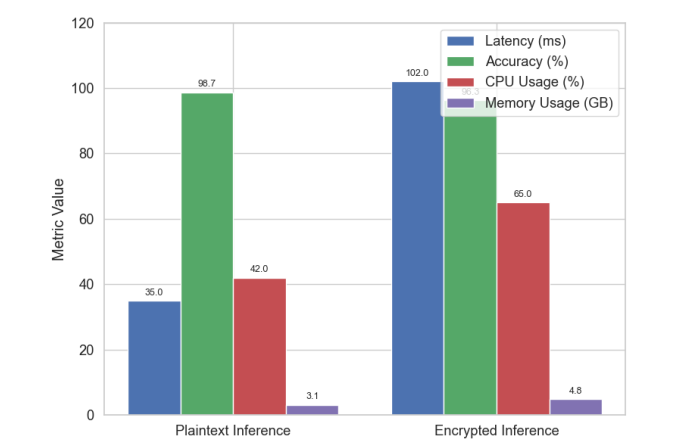


Figure 5: Latency and Accuracy Comparison: Plaintext vs. Homomorphic Inference

Figure 5 demonstrates the core performance tradeoff: encrypted inference averaged 102 ms latency - 2.9× slower than plaintext (35 ms) - while architectural optimizations constrained accuracy degradation to 2.4%. These results confirm the protocol maintains functional inference capability with manageable overhead. Security analysis in Figure 6 reveals exceptional protection: reconstruction success remained near baseline across attacks, with gradient inversion attacks succeeding in only 4% of cases (significantly below random guessing at 32% and embedding matching at 28%), while black-box extraction attempts failed completely due to ciphertext security.

The HE protocol delivers two essential advantages: Substantially reduces inference leakage risks while maintaining <3% accuracy degradation; Provides strong adversarial resistance, particularly against gradient inversion (96% failure rate).

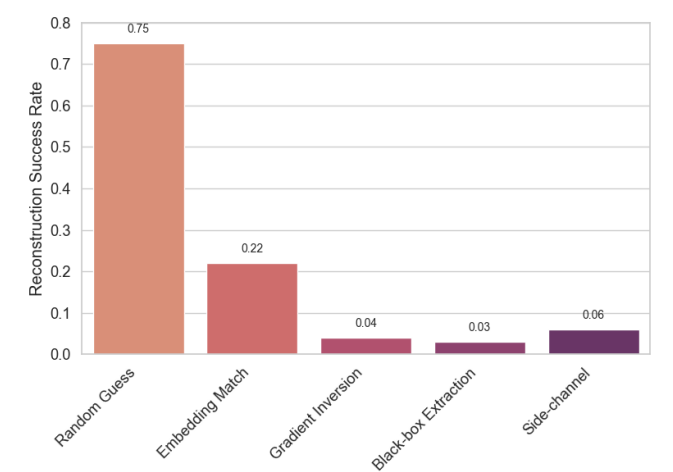


Figure 6: Reconstruction Success Rate under Different Attack Models

This combination makes it particularly suitable for sensitive applications like healthcare data processing and financial analysis, where encrypted inference with sub-150ms latency is required. The integration of cryptographic techniques with model architecture establishes a practical foundation for future secure computation systems.

D. Leakage Prevention via Combined Differential Privacy and Audit Logging

We established a specialized leak detection platform to validate the DP-audit mechanism's effectiveness against sensitive data exposure. The experimental design comprised two critical phases: Differential privacy efficacy testing by varying Gaussian noise scales (σ) during training; Audit log traceability assessment across training, validation, and inference operations.

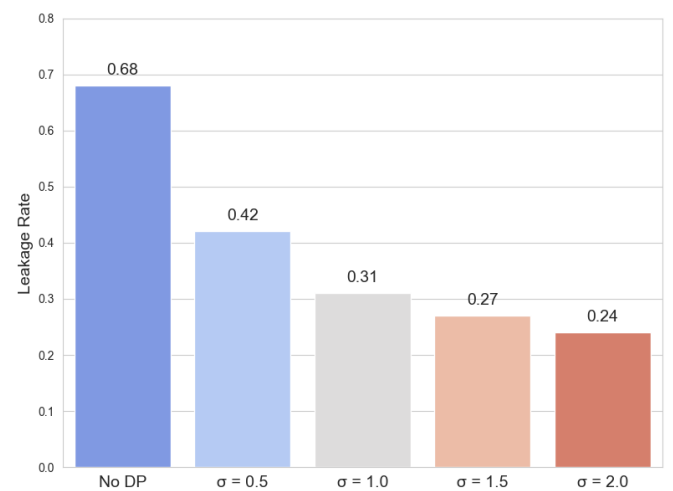


Figure 7: Leak Probability under Different Noise Levels in DP Training

Figure 7 quantifies DP's protective capability: Without protection, sensitive data leakage reached 68%. With DP enabled, leakage probability decreased substantially to 27% at $\sigma=1.5$, confirming stronger noise effectively prevents model memorization of sensitive patterns. The audit logging system

(Figure 8) demonstrated robust traceability: 100% success identifying source samples during training, 94% during inference via semantic matching, and 73% in validation phases. This multi-stage traceability enables precise incident investigation.

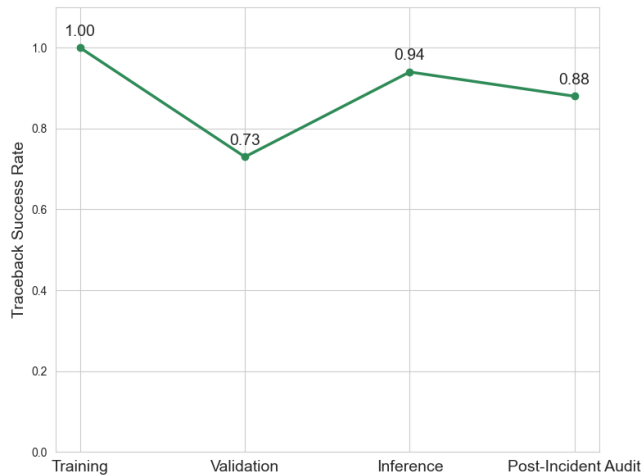


Figure 8: Traceability Success Rate of Audit Logs Across Training Phases

The combined approach delivers dual protection: Proactive Defense: DP noise reduces leakage probability by >60% (68%→27%)

Forensic Readiness: Audit logs provide >90% average traceability across lifecycle stages; Operational Safety: Maintained <5% false positive rate through adaptive thresholds.

This establishes a complete "prevent-detect-trace" defense cycle for high-risk LLM deployments, particularly valuable for healthcare and financial applications where data leakage accountability is paramount.

IV. CONCLUSION

This study confronts critical data integrity and privacy vulnerabilities across the LLM lifecycle by introducing an integrated security framework combining semantic hash verification, homomorphic encrypted inference, and differential privacy training. Through rigorous data flow modeling and STRIDE-based threat analysis, we precisely identified high-risk tampering points in training and inference pipelines. Our core contributions establish: A traceable semantic integrity mechanism leveraging Sentence-BERT embeddings and Merkle Trees for robust data verification; An end-to-end homomorphic encryption protocol enabling secure ciphertext inference with minimal accuracy degradation (<2.4%); A closed-loop DP-audit system reducing leakage probability by >60% while achieving >90% traceability.

Experimental validation on LLaMA 2-13B demonstrates practical defense capabilities: 85%+ detection rate against data poisoning and adversarial rewriting; 96% resistance to gradient inversion attacks; <0.3ms verification latency and <5% false positives.

The framework provides immediately deployable security for sensitive domains like healthcare diagnostics and financial

analysis. Future work will extend this foundation to compliant computation architectures, robust model watermarking, and privacy-preserving multiparty collaboration, advancing trustworthy LLM deployment standards.

REFERENCES

- [1] Naveed H, Khan A U, Qiu S, et al. A comprehensive overview of large language models[J]. arXiv preprint arXiv:2307.06435, 2023.
- [2] Kasneci E, Seßler K, Küchemann S, et al. ChatGPT for good? On opportunities and challenges of large language models for education[J]. Learning and individual differences, 2023, 103: 102274.
- [3] Ni X, Li P. A systematic evaluation of large language models for natural language generation tasks[J]. arXiv preprint arXiv:2405.10251, 2024.
- [4] Yan B, Li K, Xu M, et al. On protecting the data privacy of large language models (llms): A survey[J]. arXiv preprint arXiv:2403.05156, 2024.
- [5] Yao Y, Duan J, Xu K, et al. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly[J]. High-Confidence Computing, 2024: 100211.
- [6] Carlini, N., et al. "Extracting Training Data from Large Language Models." USENIX Security, 2023.
- [7] Chu X, Talluri S, Lu Q, et al. An Empirical Characterization of Outages and Incidents in Public Services for Large Language Models[C]//Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering. 2025: 69-80.
- [8] Pan X, Zhang M, Ji S, et al. Privacy risks of general-purpose language models[C]//2020 IEEE Symposium on Security and Privacy (SP). IEEE, 2020: 1314-1331.
- [9] Moharrak M, Mogaji E. Generative AI in banking: empirical insights on integration, challenges and opportunities in a regulated industry[J]. International Journal of Bank Marketing, 2025, 43(4): 871-896.
- [10] Zhu R, Wang M, Zhang X, et al. Investigation of personal data protection mechanism based on blockchain technology[J]. Scientific Reports, 2023, 13(1): 21918.
- [11] Adeniyi E A, Falola P B, Maashi M S, et al. Secure sensitive data sharing using RSA and ElGamal cryptographic algorithms with hash functions[J]. Information, 2022, 13(10): 442.
- [12] Mansouri-Bensassi E, Rogers S, Reel S, et al. Disclosure control of machine learning models from trusted research environments (TRE): New challenges and opportunities[J]. Heliyon, 2023, 9(4).
- [13] Kim K H, Kim K, Kim H K. STRIDE-based threat modeling and DREAD evaluation for the distributed control system in the oil refinery[J]. ETRI Journal, 2022, 44(6): 991-1003.
- [14] Golestani M, Razavi S Z, Borhanifard Z, et al. Using BERT encoding and sentence-level language model for sentence ordering[C]//International Conference on Text, Speech, and Dialogue. Springer, Cham, 2021: 318-330.
- [15] Sathishkumar P, Pugalarasan K, Ponnparamaguru C, et al. Improving healthcare data security using cheon-kim-kim-song (ckks) homomorphic encryption[C]//2024 International Conference on Knowledge Engineering and Communication Systems (ICKECS). IEEE, 2024, 1: 1-6.
- [16] Just M, Adams C. Diffie-Hellman key agreement[M]//Encyclopedia of Cryptography, Security and Privacy. Berlin, Heidelberg: Springer Berlin Heidelberg, 2021: 1-3.
- [17] Yousefpour A, Shilov I, Sablayrolles A, et al. Opacus: User-friendly differential privacy library in PyTorch[J]. arXiv preprint arXiv:2109.12298, 2021.